N-queens problem is to position n queens on an n x n chessboard so that no queen threatens one another. To formulate this problem and the corresponding json file, I wrote a python code nQueens.py. I adapted Qi = r where it represents “the queen in column i is in row r.” This guarantees one queen per column so they do not attack each other. Then I defined the predicate Qij, Rk, Rl that represents two queens in column i and j located at row k and l. I added [[f"Q{i}{j}", f"R{k}", f"R{l}"]] where all i, j, k, l iterate through 1 to n+1 and meet the restrictions of not located in same row or diagonal (k==l or abs(i - j) == abs(k - l)). I also made i<j restriction so that I don’t count same columns twice (e.g. Q23, Q32). For the goal clause, I want to find all the possible positions of queens, so I formulated all the combinations of Qij (Q12, Q13, Q14, Q23, Q24, Q34) and the corresponding variables matching the column index (?q1, ?q2, ?q3, ?q4). The known solutions are (?q1, ?q2, ?q3, ?q4) = (R2, R4, R1, R3) or (R3, R1, R4, R2), which matched with ResolutionProver.py solution of
{'?q1': 'R2', '?q2': 'R4', '?q3': 'R1', '?q4': 'R3'}
{'?q1': 'R3', '?q2': 'R1', '?q3': 'R4', '?q4': 'R2'}